```
        exceptionHandling.authenticationEntryPoint(                    ❶
          new HttpStatusServerEntryPoint(HttpStatus.UNAUTHORIZED)))
      .oauth2Login(Customizer.withDefaults())
      .logout(logout -> logout.logoutSuccessHandler(
      oidcLogoutSuccessHandler(clientRegistrationRepository)))
      .build();
  }
}
```

❶ When an exception is thrown because a user is not authenticated, it replies with an HTTP 401 response.

At this point, the Angular application can explicitly intercept HTTP `401` responses and trigger the authentication flow. However, since the SPA is now in charge of starting the flow, we need to permit unauthenticated access to its static resources. We would also like to retrieve the books in the catalog without being authenticated, so let's permit `GET` requests to the `/books/**` endpoints as well. Go ahead and update the `SecurityWebFilterChain` bean in the `SecurityConfig` class as follows.

Listing 11.15 Allowing unauthenticated `GET` requests to SPA and books

```
@EnableWebFluxSecurity
public class SecurityConfig {
  ...

  @Bean
  SecurityWebFilterChain springSecurityFilterChain(
    ServerHttpSecurity http,
    ReactiveClientRegistrationRepository clientRegistrationRepository
  ) {
    return http
      .authorizeExchange(exchange -> exchange
        .pathMatchers("/", "/*.css", "/*.js", "/favicon.ico")
          .permitAll()                                                 ❶
        .pathMatchers(HttpMethod.GET, "/books/**")
          .permitAll()                                                 ❷
        .anyExchange().authenticated()                                 ❸
      )
      .exceptionHandling(exceptionHandling -> exceptionHandling
        .authenticationEntryPoint(
        new HttpStatusServerEntryPoint(HttpStatus.UNAUTHORIZED)))
```